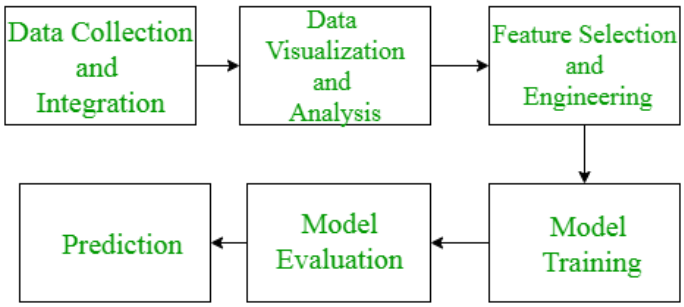
Lecture 17-12-2023: Machine Learning Basics

Machine learning:

Machine learning is a branch of artificial intelligence (AI) that focuses on developing algorithms and statistical models that enable computers to learn from and make predictions or decisions based on data.

Steps:



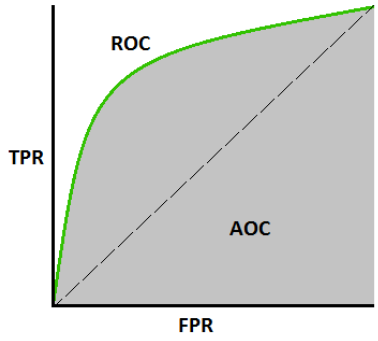
Evaluation measure:

By ANC

AUC: Area under the ROC curve

AUC - ROC curve is a performance measurement for the classification problems at various threshold settings. ROC is a probability curve and AUC represents the degree or measure of separability.

The ROC curve is plotted with TPR against the FPR where TPR is on the y-axis and FPR is on the x-axis.



TPR is nothing but the recall

$$\text{Specificity} = \frac{TN}{TN + FP}$$

$$FPR = 1 - \text{Specificity}$$

The Higher the AUC, the better the model

09/01/24: Clustering

Clustering is a type of unsupervised learning wherein data points are grouped into different sets based on their degree of similarity.

The various **types** of clustering are:

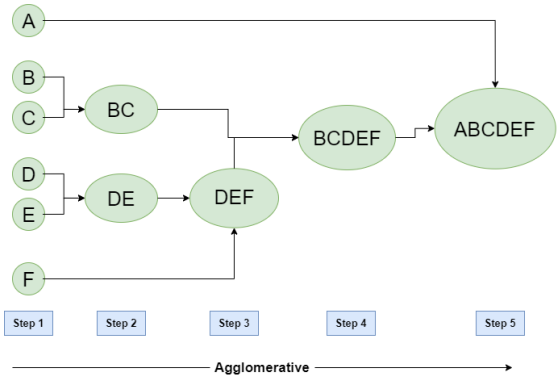
- Hierarchical clustering
  - Agglomerative clustering
  - Divisive clustering

- Partitioning clustering
  - K-Means clustering
  - Fuzzy C-Means clustering

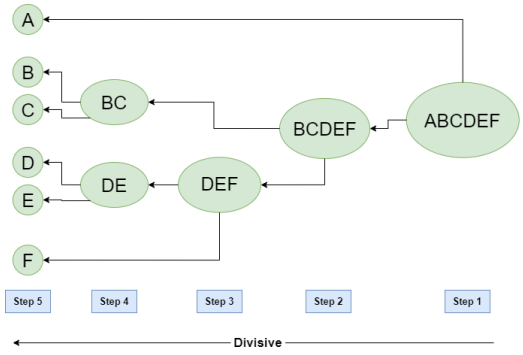
Hierarchical clustering:

*Hierarchical clustering uses a tree-like structure,

Agglomerative hierarchical clustering begins with each data point as a separate cluster and iteratively merges the closest pairs of clusters until all data points belong to a single cluster or a desired number of clusters is reached.

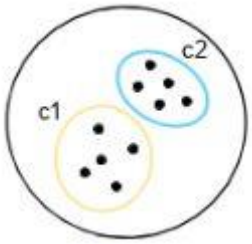


Divisive hierarchical clustering begins with all data points in a single cluster and recursively divides them into smaller clusters until each data point belongs to its own cluster or a desired number of clusters is reached.

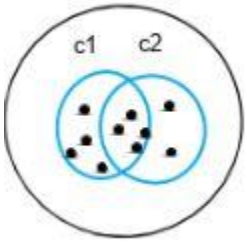


Partitioning Clustering:
Partitioning clustering is split into two subtypes - K-Means clustering and Fuzzy C-Means.

In **k-means** clustering, the objects are divided into several clusters mentioned by the number 'K.' So if we say K = 2, the objects are divided into two clusters, c1 and c2, as shown:



Fuzzy c-means is very similar to k-means in the sense that it clusters objects that have similar characteristics together. In k-means clustering, a single object cannot belong to two different clusters. But in c-means, objects can belong to more than one cluster, as shown.



Distance measure:

<u>i) Manhattan Distance:</u>

If A(p1, p2) and B(q1, q2) are two points:

$$d = |p_1 - q_1| + |p_2 - q_2|$$

<u>ii) Euclidean Distance:</u>

$$d = ((p_1 - q_1)^2 + (p_2 - q_2)^2)^{1/2}$$

<u>iii) Cosine Distance:</u>

$$\cos\theta = \frac{\vec{a}\cdot\vec{b}}{\|\vec{a}\|\|\vec{b}\|}$$

$$\|\vec{a}\| = \sqrt{a_1^2 + a_2^2 + a_3^2 + \cdots + a_n^2}$$

$$\|\vec{b}\| = \sqrt{b_1^2 + b_2^2 + b_3^2 + \cdots + b_n^2}$$

Cosine distance = 1- Cosine similarity

K-means Algorithm:

- Choose the number of clusters K.
- Randomly select any K data points as cluster centers.
  Select cluster centers in such a way that they are as farther as possible from each other.
- Calculate the distance between each data point and each cluster center.
- Assign each data point to some cluster.
  A data point is assigned to that cluster whose center is nearest to that data point.
- Re-compute the center of newly formed clusters.
  The center of a cluster is computed by taking mean of all the data points contained in that cluster.
- Keep repeating the procedure from Step-03 to Step-05 until any of the following stopping criteria is met-
  - Center of newly formed clusters do not change
  - Data points remain present in the same cluster
  - Maximum number of iterations are reached

Problem 1:
Cluster the following eight points into three clusters:
A1(2, 10), A2(2, 5), A3(8, 4), A4(5, 8), A5(7, 5), A6(6, 4), A7(1, 2), A8(4, 9).

Solution:
Let's use Manhattan distance
Selected centroids: A1(2,10), A4(5,8), A7(1,2)
Iteration 1:

Cluster-01	Cluster-02	Cluster-03
• A1(2, 10)	• A3(8, 4) • A4(5, 8) • A5(7, 5) • A6(6, 4) • A8(4, 9)	• A2(2, 5) • A7(1, 2)
A1(2, 10)	((8 + 5 + 7 + 6 + 4)/5, (4 + 8 + 5 + 4 + 9)/5) = (6, 6)	((2 + 1)/2, (5 + 2)/2) = (1.5, 3.5)

Given Points	Distance from center (2, 10) of Cluster-01	Distance from center (5, 8) of Cluster-02	Distance from center (1, 2) of Cluster-03	Point belongs to Cluster
A1(2, 10)	0	5	9	C1
A2(2, 5)	5	6	4	C3
A3(8, 4)	12	7	9	C2
A4(5, 8)	5	0	10	C2
A5(7, 5)	10	5	9	C2
A6(6, 4)	10	5	7	C2
A7(1, 2)	9	10	0	C3
A8(4, 9)	3	2	10	C2

Iteration 2:

Given Points	Distance from center (2, 10) of Cluster-01	Distance from center (6, 6) of Cluster-02	Distance from center (1.5, 3.5) of Cluster-03	Point belongs to Cluster
A1(2, 10)	0	8	7	C1
A2(2, 5)	5	5	2	C3
A3(8, 4)	12	4	7	C2
A4(5, 8)	5	3	8	C2
A5(7, 5)	10	2	7	C2
A6(6, 4)	10	2	5	C2
A7(1, 2)	9	9	2	C3
A8(4, 9)	3	5	8	C1

Cluster-01	Cluster-02	Cluster-03
• A1(2, 10) • A8(4, 9)	• A3(8, 4) • A4(5, 8) • A5(7, 5) • A6(6, 4)	• A2(2, 5) • A7(1, 2)

Disadvantages of K-Means:

1. **Sensitivity to initial centroids**: K-means is sensitive to the initial selection of centroids and can converge to a suboptimal solution.
2. **Requires specifying the number of clusters**: The number of clusters k needs to be specified before running the algorithm, which can be challenging in some applications.
3. **Sensitive to outliers:** K-means is sensitive to outliers, which can have a significant impact on the resulting clusters.

Elbow Method

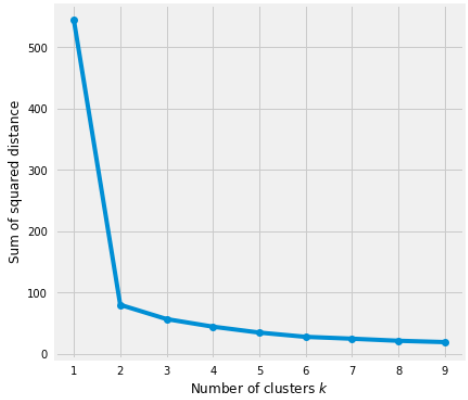
The Elbow method is one of the most popular ways to find the optimal number of clusters. This method uses the concept of WCSS value. **WCSS** stands for **Within Cluster Sum of Squares**, which defines the total variations within a cluster. The formula to calculate the value of WCSS (for 3 clusters) is given below

$$WCSS= \sum_{P_i \text{ in Cluster1}} distance(P_i\ C_1)^2 + \sum_{P_i \text{ in Cluster2}} distance(P_i\ C_2)^2 + \sum_{P_i \text{ in CLuster3}} distance(P_i\ C_3)^2$$

Sum of the square of the distances between each data point and its centroid within a cluster1 and the same for the other two terms.

To find the optimal value of clusters, the elbow method follows the below steps:

- It executes the K-means clustering on a given dataset for different K values (ranges from 1-10).
- For each value of K, calculates the WCSS value.
- Plots a curve between calculated WCSS values and the number of clusters K.
- The sharp point of bend or a point of the plot looks like an arm, then that point is considered as the best value of K.



[1] https://utsavdesai26.medium.com/the-beginners-guide-to-clustering-in-machine-learning-331987a7ceaf
[2] https://www.javatpoint.com/k-means-clustering-algorithm-in-machine-learning

21/01/2024: KNN-K-nearest neighbors

Step #1 - Assign a value to **K**.
Step #2 - Calculate the distance between the new data entry and all other existing data entries (you'll learn how to do this shortly). Arrange them in ascending order.
Step #3 - Find the **K** nearest neighbors to the new entry based on the calculated distances.
Step #4 - Assign the new data entry to the majority class in the nearest neighbors.

Problem 01:

BRIGHTNESS	SATURATION	CLASS
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue
60	10	Red
25	80	Blue

Let's the value of **K** is 5. Find out the class that the datapoint (20, 35) belongs to.

Solution:
Here we use Euclidean distance as the distance measure.

$d1 = \sqrt{(20 - 40)^2 + (35 - 20)^2}$
$= \sqrt{400 + 225}$
$= \sqrt{625}$
$= 25$

BRIGHTNESS	SATURATION	CLASS	DISTANCE
40	20	Red	25
50	50	Blue	33.54
60	90	Blue	68.01
10	25	Red	10
70	70	Blue	61.03
60	10	Red	47.17
25	80	Blue	45

Let's rearrange the distances in ascending order:

CLASS	DISTANCE
Red	10
Red	25
Blue	33.54
Blue	45
Red	47.17
Blue	61.03
Blue	68.01

Since we chose 5 as the value of **K**, we'll only consider the first five rows.
As you can see above, the majority class within the 5 nearest neighbors to the new entry is **Red**. Therefore, we'll classify the new entry as **Red**.

[1] https://www.freecodecamp.org/news/k-nearest-neighbors-algorithm-classifiers-and-model-example/

18/02/2024: Linear Regression

Linear regression is used to predict the relationship between two variables by applying a linear equation to observed data.
The measure of the relationship between two variables is shown by the correlation coefficient. The range of the coefficient lies between -1 to +1.

Linear Regression Formula

Linear Regression Formula is given by the equation

$$Y = a + bX$$

We will find the value of a and b by using the below formula:

$$a = \frac{(\sum Y)(\sum X^2) - (\sum X)(\sum XY)}{n(\sum x^2) - (\sum x)^2}$$

$$b = \frac{n(\sum XY) - (\sum X)(\sum Y)}{n(\sum x^2) - (\sum x)^2}$$

Problem 01:

Find a linear regression equation for the following two sets of data:

x	2	4	6	8
y	3	7	5	10

Solution:

Construct the table and find the value

x	**y**	**x^2**	**xy**
2	3	4	6
4	7	16	28
6	5	36	30
8	10	64	80
$\Sigma x = 20$	$\Sigma y = 25$	$\Sigma x^2 = 120$	$\Sigma xy = 144$

a= $$\frac{(\sum Y)(\sum X^2)-(\sum X)(\sum XY)}{n(\sum x^2)-(\sum x)^2}$$

Now put the values in the equation

$$a=\frac{25\times 120-20\times 144}{4\times 120-400}$$

a= $$\frac{120}{80}$$

a=1.5

b= $$\frac{n(\sum XY)-(\sum X)(\sum Y)}{n(\sum x^2)-(\sum x)^2}$$

Put the values in the equation

$$b=\frac{4\times 144-20\times 25}{4\times 120-400}$$

b= $$\frac{76}{80}$$

b=0.95

Hence we got the value of a = 1.5 and b = 0.95

The linear equation is given by

Y = a + bx

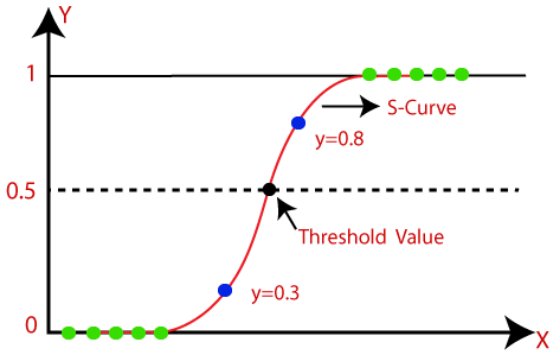
Now put the value of a and b in the equation

Hence equation of linear regression is y = 1.5 + 0.95x

[1] https://www.vedantu.com/maths/linear-regression

25/02/2024: Logistic Regression

In Logistic regression, instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1). The sigmoid function is a mathematical function used to map the predicted values to probabilities.



The Logistic regression equation can be obtained from the Linear Regression equation. The mathematical steps to get Logistic Regression equations are given below:

- We know the equation of the straight line can be written as:

$$y= b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \cdots + b_nx_n$$

- In Logistic Regression y can be between 0 and 1 only, so for this let's divide the above equation by (1-y):

$$\frac{y}{1-y}$$ **; 0 for y= 0, and infinity for y=1**

- But we need range between -[infinity] to +[infinity], then take logarithm of the equation it will become:

$$\boldsymbol{log\left[\frac{y}{1-y}\right] = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \cdots + b_nx_n}$$

The above equation is the final equation for Logistic Regression.

Type of Logistic Regression:
On the basis of the categories, Logistic Regression can be classified into three types:

- **Binomial:** In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.

- **Multinomial:** In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as "cat", "dogs", or "sheep"

- **Ordinal:** In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as "low", "Medium", or "High".

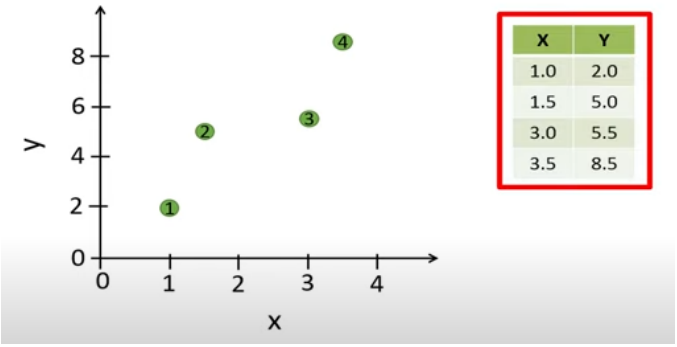
Linear vs logistic regression:

Linear Regression	**Logistic Regression**
Linear regression is used to predict the **continuous** dependent variable using a given set of independent variables.	Logistic Regression is used to predict the **categorical** dependent variable using a given set of independent variables.
Linear Regression is used for solving **Regression** problem.	Logistic regression is used for solving **Classification** problems.
In linear regression, we find the **best fit line**, by which we can easily predict the output.	In Logistic Regression, we find the **S-curve** by which we can classify the samples.
Least square estimation method is used for estimation of accuracy.	**Maximum likelihood** estimation method is used for estimation of accuracy.
Relationship between dependent variable and independent variable must be **linear**.	It is **not required** to have the linear relationship between the dependent and independent variable.

[1] https://www.javatpoint.com/logistic-regression-in-machine-learning
[2] https://www.javatpoint.com/linear-regression-vs-logistic-regression-in-machine-learning

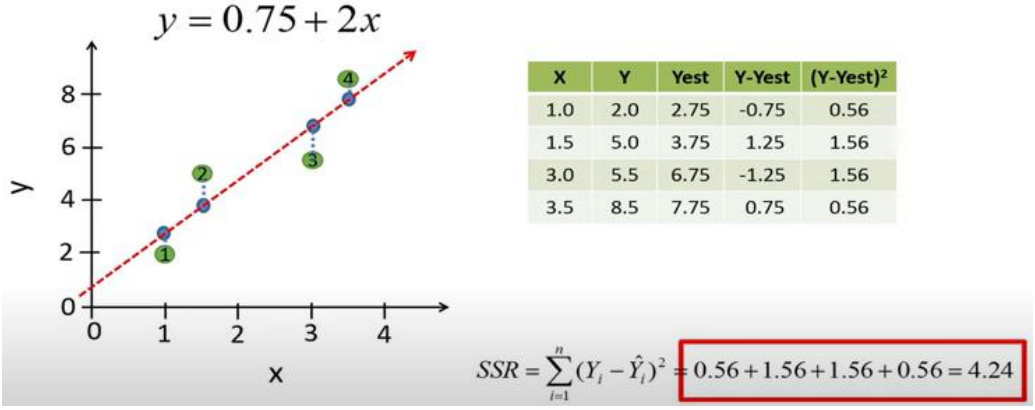
OLS Vs MLE

Remember that the aim of simple linear regression is to fit the line that fits as good as possible to the data. To fit the line to some data we can either use ordinary least squares or the maximum likelihood method. To explain how the two methods work we'll use the following data set.



X	Y
1.0	2.0
1.5	5.0
3.0	5.5
3.5	8.5

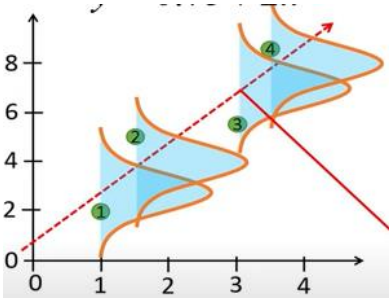
Let's fit an arbitrary line to the data with an intercept of 0.75 and a slope of 2. Based on this equation we can calculate estimated y values. We can now calculate the so called residuals which are the vertical distances between each data point and the line. This distance is calculated by subtracting the estimated y values from the observed y values. A negative residual means that the data point is below the line whereas a positive residual means that the data point is above the line. If we square the residuals we'll get the so-called squared residuals. If you sum those squared residuals we'll get the so-called sum or squared residuals.



X	Y	Yest	Y-Yest	(Y-Yest)²
1.0	2.0	2.75	-0.75	0.56
1.5	5.0	3.75	1.25	1.56
3.0	5.5	6.75	-1.25	1.56
3.5	8.5	7.75	0.75	0.56

$$SSR = \sum_{i=1}^{n} (Y_i - \hat{Y}_i)^2 = 0.56 + 1.56 + 1.56 + 0.56 = 4.24$$

In contrast to the ordinary squares the maximum likelihood method finds the optimal values of the intercept and slope of the line that maximizes the likelihood.
Instead of calculating the distances between the data points and a line the maximum likelihood method can be seen as we place a curve based on some distribution around the line. We will use the normal distribution as an example. Where mu is the mean and sigma is the standard deviation. Since the curves are centered around the line, the value mu corresponds to the y value of the line the; estimated y values.



We can therefore replace mu by the equation of the line.

$$f(Y) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(Y-\mu)^2}{2\sigma^2}}$$

$$f(Y) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(Y_i-(\beta_0+\beta_1 x_i))^2}{2\sigma^2}}$$

Sigma determines the width of the curve and is estimated based on the spread of the data points around the line.

The likelihood function:

The likelihood function is defined as the joint probability density (or mass) function of Y1, Y2,…,Yn (of our observations) as a function of theta

$$L(\theta) = \prod_{i=1}^{n} f(Y_i;\theta)$$

Where theta is a vector of parameters.

For example, if we like to estimate the intercept slope and the standard deviation of some data that we think is normally distributed, we would simply use the probability density function of the normal distribution.

For every data point we would calculate the corresponding likelihood value that in this case corresponds to the height of the normal distribution and multiply all those likelihoods so that we get the joint likelihood of the estimated parameters.

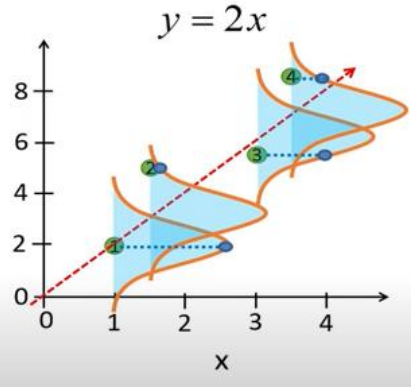
$$L(\hat{\beta}_0, \hat{\beta}_1, \hat{\sigma}) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\hat{\sigma}^2}} e^{-\frac{(Y_i-(\hat{\beta}_0+\hat{\beta}_1 x_i))^2}{2\hat{\sigma}^2}}$$

Let's calculate the joint likelihood by setting the intercept to 0.75 and the slope to 2.

X	Y	Yest	Y-Yest	(Y-Yest)²	L
1.0	2.0	2.75	-0.75	0.56	0.301
1.5	5.0	3.75	1.25	1.56	0.183
3.0	5.5	6.75	-1.25	1.56	0.183
3.5	8.5	7.75	0.75	0.56	0.301

$L(\hat{\beta}_0=0.75,\hat{\beta}_1=2,\hat{\sigma}=1)=$ $0.301\cdot0.183\cdot0.183\cdot0.301=0.003$

Let's change the intercept of the line from 0.75 to zero.

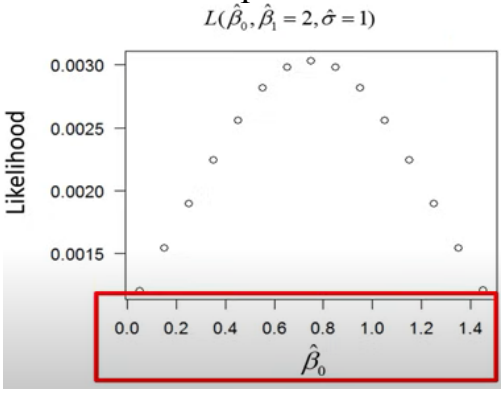


X	Y	Yest	Y-Yest	(Y-Yest)²	L
1.0	2.0	2	0	0	0.399
1.5	5.0	3	2	4	0.054
3.0	5.5	6	-0.5	0.25	0.352
3.5	8.5	7	1.5	2.25	0.130

$SSR=\sum_{i=1}^{n}(Y_i-\hat{Y}_i)^2=0+4+0.25+2.25=6.5$

We see that the sum of the squared residuals has increased from 4.35 to 6.5. Which means that this line fits worse to the data compared to our previous line with an intercept of 0.75. When we calculate the joint likelihood value we see that it has been reduced from 0.003 to 0.001. A reduced likelihood value therefore means the line fits worse to the data.
It is therefore more likely that the intercept of the line is equal to 0.75 than 0 given the data.

Let's try many different values of intercept.



SSE curve is reverse of above
[1] https://www.youtube.com/watch?v=bhTIpGtWtzQ

Bayesian Belief Network

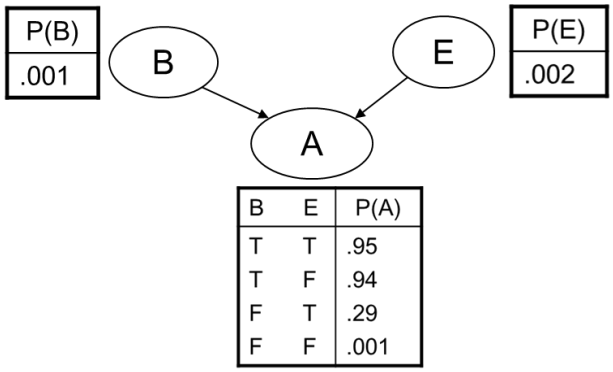
Question 03: What is Bayesian Network? Describe the syntax and semantic of Bayesian network.

Answer:

"A Bayesian network is a probabilistic graphical model which represents a set of variables and their conditional dependencies using a directed acyclic graph."

Syntax:

- Each node corresponds to a random variable
- If there is an edge from node X to node Y, X is said to be a parent of Y. It is a DAG
- Each node Xi has a conditional probability distribution P(Xi | Parents(Xi))that quantifies the effect of the parents on the node.



Semantic:

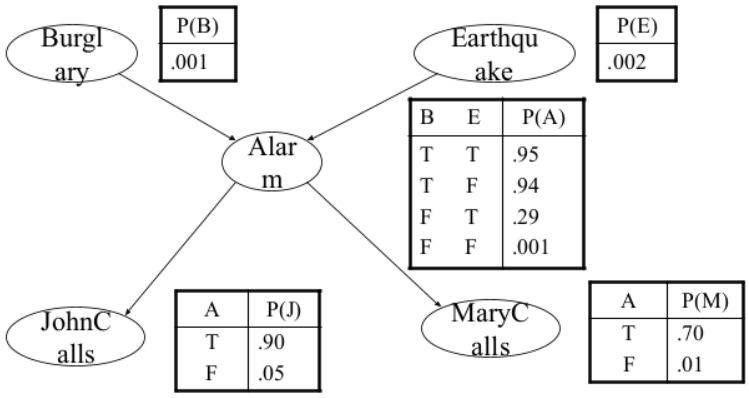
- **$P(X_1, X_2, \cdots, X_n)$**
  $= P(X_n \mid X_{n-1}, \cdots, X_1).P(X_{n-1} \mid X_{n-2}, \cdots, X_1). \cdots .P(X_2 \mid X_1).P(X_1)$
  $= \Pi_i P(X_i \mid X_{i-1}, \cdots, X_1)$
- $P(X_i \mid X_{i-1}, \cdots, X_1) = P(X_i \mid Parents(X_i))$

$$P(A,B,E) = P(A|B\wedge E).P(B).P(E).$$

Markov Condition: Given its parents (P1, P2) node x is conditionally independent of its non-descendent (ND1, ND2)

Example:

$P (J \wedge M \wedge A \wedge \neg B \wedge \neg E)$
$= P (J \mid A).P (M \mid A).P(A \mid \neg B \wedge \neg E).P(\neg B).P(\neg E)$
$= 0.00062$



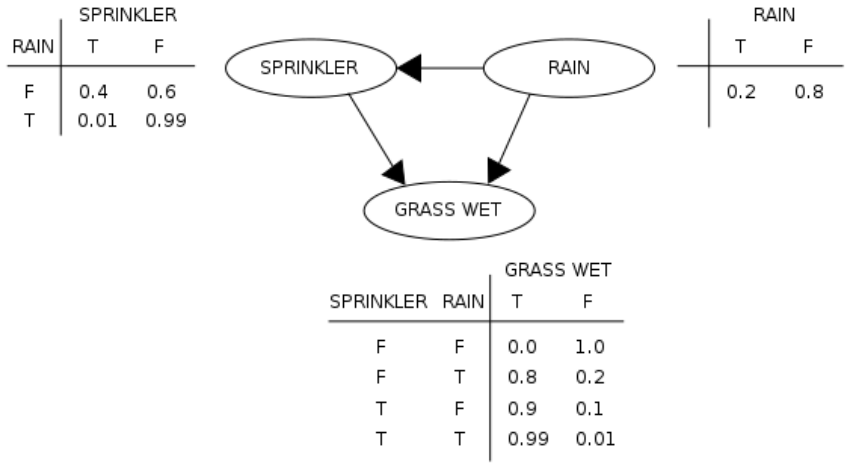
* Using Bayesian Network to calculate probability is called inference
* In general, inference involves queries of the form:

P(X|E)

E = Evidence variable(s)

X = Query variable(s)

Example:



What is the probability that it is raining given the grass is wet?

$$P(G=T,S=T,R=T)$$
$$= P(G=T|S=T,R=T)P(S=T|R=T)P(R=T)$$
$$= 0.99 \times 0.01 \times 0.2$$
$$= 0.00198.$$

$$P(R=T \mid G=T) = \frac{P(G=T,R=T)}{P(G=T)} = \frac{\sum_{S\in\{T,F\}} P(G=T,S,R=T)}{\sum_{S,R\in\{T,F\}} P(G=T,S,R)}$$

$$= \frac{(0.99 \times 0.01 \times 0.2 = 0.00198_{TTT}) + (0.8 \times 0.99 \times 0.2 = 0.1584_{TFT})}{0.00198_{TTT} + 0.288_{TTF} + 0.1584_{TFT} + 0_{TFF}} \approx 35.77\%.$$

Naïve bias classifier: By ANC

Bayes Theorem:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Given the following dataset.

	Outlook	Temperature	Humidity	Windy	Play Golf
0	Rainy	Hot	High	False	No
1	Rainy	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Sunny	Mild	High	False	Yes
4	Sunny	Cool	Normal	False	Yes
5	Sunny	Cool	Normal	True	No
6	Overcast	Cool	Normal	True	Yes
7	Rainy	Mild	High	False	No
8	Rainy	Cool	Normal	False	Yes
9	Sunny	Mild	Normal	False	Yes
10	Rainy	Mild	Normal	True	Yes
11	Overcast	Mild	High	True	Yes
12	Overcast	Hot	Normal	False	Yes
13	Sunny	Mild	High	True	No

Calculate play or not for: today = (Sunny, Hot, Normal, False)

Answer:

Outlook

	Yes	No	P(yes)	P(no)
Sunny	3	2	2/9	3/5
Overcast	4	0	4/9	0/5
Rainy	3	2	3/9	2/5
Total	9	5	100%	100%

Temperature

	Yes	No	P(yes)	P(no)
Hot	2	2	2/9	2/5
Mild	4	2	4/9	2/5
Cool	3	1	3/9	1/5
Total	9	5	100%	100%

Humidity

	Yes	No	P(yes)	P(no)
High	3	4	3/9	4/5
Normal	6	1	6/9	1/5
Total	9	5	100%	100%

Wind

	Yes	No	P(yes)	P(no)
False	6	2	6/9	2/5
True	3	3	3/9	3/5
Total	9	5	100%	100%

Play		P(Yes)/P(No)
Yes	9	9/14
No	5	5/14
Total	14	100%

*** P (sunny | yes) = 3/9

$$P\ (Yes \mid today) = \frac{\text{P(SunnyOutlook|Yes)P(HotTemperature|Yes)P(NormalHumidity|Yes)P(NoWind|Yes)P(Yes)}}{\text{P(today)}}$$

$$P\ (No \mid today) = \frac{\text{P(SunnyOutlook|No)P(HotTemperature|No)P(NormalHumidity|No)P(NoWind|No)P(No)}}{\text{P(today)}}$$

Since, P(today) is common in both probabilities, we can ignore P(today) and find proportional probabilities as:

$$P(Yes|today) \propto \frac{3}{9}.\frac{2}{9}.\frac{6}{9}.\frac{6}{9}.\frac{9}{14} \approx 0.02116$$

and

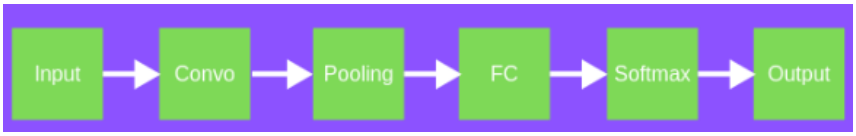
$$P(No|today) \propto \frac{3}{5}.\frac{2}{5}.\frac{1}{5}.\frac{2}{5}.\frac{5}{14} \approx 0.0068$$

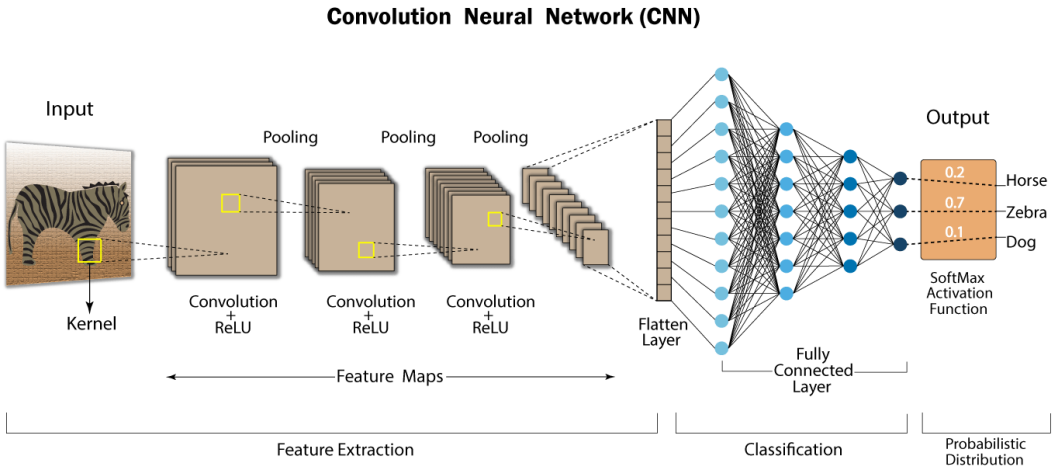
<u>Why Naïve bias classifier:</u>

ANC Note

Basics of CNN in deep learning

Different layers in CNN:





1. **Input Layer:** The input layer is where the raw data is fed into the network. In the context of image classification, this would be the pixel values of the image.

2. **Convolution Layer:** Convolutional layers apply filters (kernels) to the input data to detect features such as edges, textures, or patterns. These filters slide over the input image, performing element-wise multiplication and summing up the results to produce feature maps.

3. **Pooling Layer:** Pooling layers reduce the spatial dimensions (width and height) of the feature maps while retaining the most important information. Max pooling and average pooling are common pooling operations, where the maximum or average value within a window is retained, respectively.

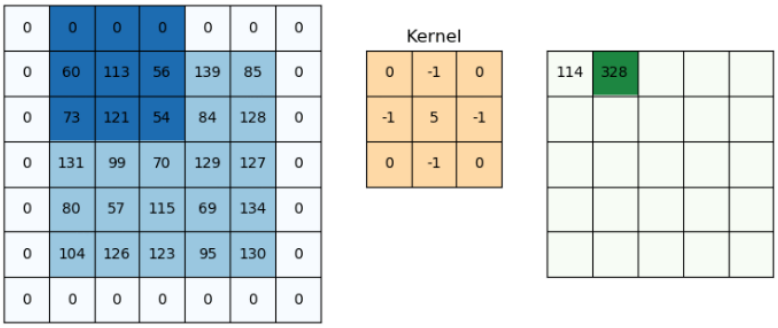
4. **Fully Connected Layer:** Fully connected layers are traditional neural network layers where each neuron is connected to every neuron in the previous and subsequent layers. They take the flattened output of the convolutional and pooling layers and perform complex transformations to learn high-level features and relationships in the data.

5. **Softmax Layer:** The softmax layer is typically the final layer in a CNN used for classification tasks. It applies the softmax function to the output of the

previous layer, converting the raw scores (logits) into probabilities. Each value represents the likelihood of the input belonging to a particular class.

6. **Output Layer:** The output layer produces the final predictions or classifications based on the probabilities calculated by the softmax layer

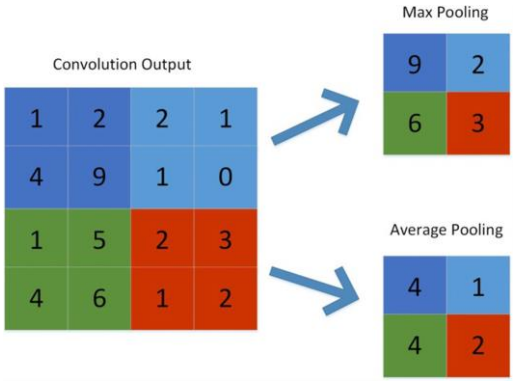
Convolution layer:



Why padding?

Without padding, as the filter moves across the input image, pixels at the edges receive fewer operations compared to those in the center. This can result in information loss and reduced effectiveness in detecting features, especially near the edges of the image. Padding ensures that every pixel in the input image has an equal opportunity to contribute to the feature maps.

Pooling:



ReLU:

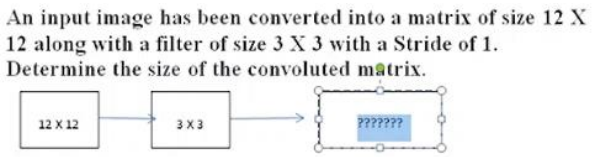
The ***rectified linear activation function***, or ReLU for short, is a piecewise linear function that, if the input is positive, outputs the input directly; else, it outputs zero. Because a model that utilizes it is quicker to train and generally produces higher performance

$$Y = \max(X, 0)$$

Convoluted matrix size (Also for Pooling):

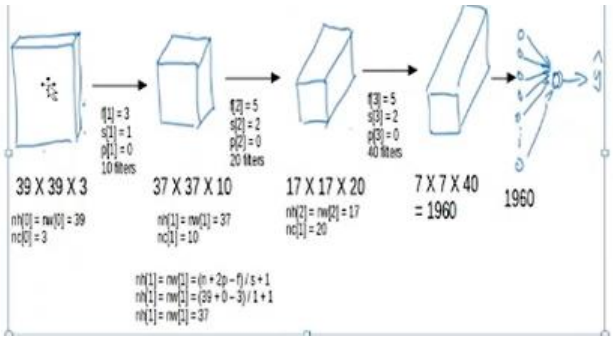
$$C = ((n-f+2p)/s) + 1$$

n = size of input matrix
f = size of filter matrix
p = padding
s = stride



C = ((12-3+2*0)/1)+1
= 10

Size of the convolution matrix is 10 X 10



* keras.layers.Conv2D(# filters, kernel_size, strides=(1, 1), padding="valid",…)
Padding = valid: no padding
Padding = same: padding size = (filter size−1)/2
When padding="same" and strides=1, the output has the same size as the input.

* keras.layers.AveragePooling2D(pool_size, strides=None, padding="valid", …)
Stride = None: it will default to pool_size

Calculating number of parameters in CNN:

Parameters in general are weights that are learnt during training. They are weight matrices that contribute to model's predictive power, changed during back-propagation process.

* First calculate output shape of each layer

Input layer: parameters = 0.

CONV layer: ((m * n * d) + 1)* k

((shape of width of the filter * shape of height of the filter * number of filters in the previous layer+1)*number of filters).

POOL layer: parameters = 0.

Fully Connected Layer (FC):

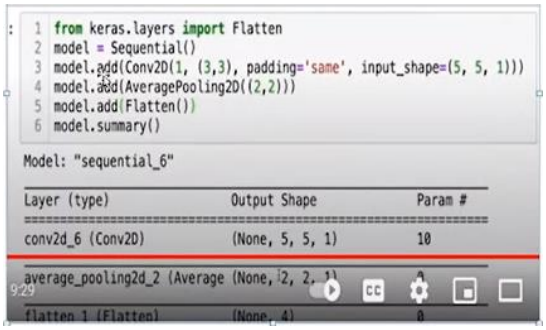
((current layer neurons c * previous layer neurons p) + (1*c).

SL.No		Activation Shape	Activation Size	# Parameters
1.	Input Layer:	(32, 32, 3)	3072	0
2.	CONV1 (f=5, s=1)	(28, 28, 8)	6272	608
3.	POOL1	(14, 14, 8)	1568	0
4.	CONV2 (f=5, s=1)	(10, 10, 16)	1600	3216
5.	POOL2	(5, 5, 16)	400	0
6.	FC3	(120, 1)	120	48120
7.	FC4	(84, 1)	84	10164
8.	Softmax	(10, 1)	10	850

```
1 my_input = Input(shape = (100, 100, 3))
2 my_output = Convolution2D(8, 3)(my_input)
3 my_output = Activation('sigmoid')(my_output)
4 my_output = Convolution2D(1, 3)(my_output)
5 my_output = Activation('sigmoid')(my_output)
6 model = Model(my_input, my_output)
```

```
1 model.summary()
```

```
Layer (type)                 Output Shape              Param #
=================================================================
input_20 (InputLayer)        [(None, 100, 100, 3)]     0
_________________________________________________________________
conv2d_24 (Conv2D)           (None, 98, 98, 8)         224
_________________________________________________________________
activation_24 (Activation)   (None, 98, 98, 8)         0
_________________________________________________________________
conv2d_25 (Conv2D)           (None, 96, 96, 1)         73
_________________________________________________________________
activation_25 (Activation)   (None, 96, 96, 1)         0
=================================================================
Total params: 297
Trainable params: 297
Non-trainable params: 0
```



Dropout Layer:
The Dropout layer is a regularization technique used in neural networks, to prevent overfitting and improve generalization performance.

During training, Dropout randomly sets a fraction of input units (neurons) to zero with a specified probability (dropout rate). This means that the neurons are temporarily "dropped out" of the network, along with all their connections, during training.

Sigmoid:

$$\sigma(x) = \frac{1}{1+e^{-x}} = \frac{e^x}{1+e^x}$$

e = Euler's number

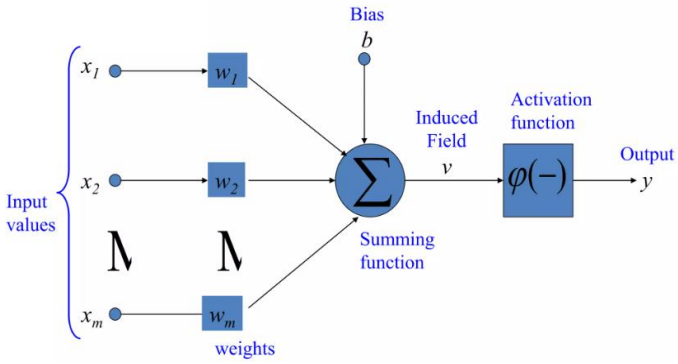
Softmax:

$$s\left(x_i\right) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

[1] https://discuss.boardinfinity.com/t/explain-the-different-layers-in-cnn/8535
[2] https://www.linkedin.com/pulse/what-convolutional-neural-network-cnn-deep-learning-nafiz-shahriar/
[3] https://medium.com/@draj0718/convolutional-neural-networks-cnn-architectures-explained-716fb197b243
[4] https://towardsdatascience.com/understanding-and-calculating-the-number-of-parameters-in-convolution-neural-networks-cnns-fc88790d530d
[5] https://www.youtube.com/watch?v=bikmA-VmSbY&t=375s

Neural Network

• Artificial neural network (ANN) is a machine learning approach that models human brain and consists of a number of artificial neurons.
A neuron is the most fundamental unit of processing. It's also called a perceptron.



A Neuron

Activation Function:

- step function: $\varphi(x) = \begin{cases} 0 & \text{if } x < \theta \\ 1 & \text{if } x > \theta \end{cases}$
- sigmoid function $\varphi(x) = \frac{1}{1 + \exp(-x)}$
- Gaussian function: $\varphi(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right)$

Three different classes of neural architecture:

- Single layer feed forward
- Multi-layer feed forward
- Recurrent

Perceptron for classification:

• Perceptron is special form of **single** layer feed forward

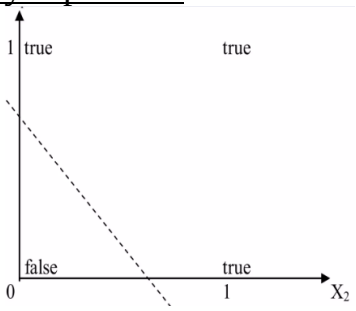
• The perceptron is used for **binary** classification.

• First train a perceptron for a classification task.
  – Find suitable weights in such a way that the training examples are correctly classified.

• The perceptron can only model linearly separable classes. When the two classes are not linearly separable, it may be desirable to obtain a linear separator that minimizes the mean squared error.

• Given training examples of classes C1, C2 train the perceptron in such a way that:
  – If the output of the perceptron is +1 then the input is assigned to class C1 If the output is -1 then the input is assigned to C2

Boolean Function OR-Linearly separable:



Learning process for perceptron:

- Initially assign random weights to inputs between -0.5 and +0.5
- Training data is presented to perceptron and its output is observed.
- If output is incorrect, the weights are adjusted accordingly using following formula.

  $wi \leftarrow wi + (a * xi * e)$, where 'e' is error produced and 'a' ($-1 < a < 1$) is learning rate

  - 'a' is defined as 0 if output is correct, it is +ve, if output is too low and –ve, if output is too high.
  - Once the modification to weights has taken place, the next piece of training data is used in the same way.
  - Once all the training data have been applied, the process starts again until all the weights are correct and all errors are zero.
  - Each iteration of this process is known as an epoch.

* Perceptron uses a step function that return +1 (positive class) if weighted sum of its input is >0. Otherwise return -1 (Negative class)

Example: OR Function:

Let w1 = -0.2, w2 = 0.4, a = 0.2,

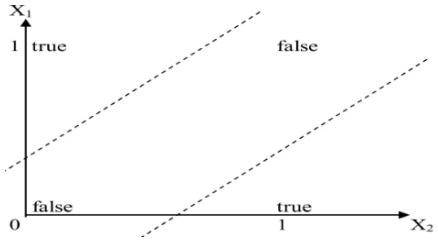
X1	X2	T	Y	W1	W2
0	0	0	0	-0.2	0.4
0	1	1	1	-0.2	0.4
1	0	1	0	-0.2 + (0.2*1*1) = 0	0.4 + (0.2*0*1) = 0.4
1	1	1	1	0	0.4
0	0	0	0	0	0.4
0	1	1	1	0	0.4
1	0	1	0	0 + (0.2*1*1) = 0.2	0.4 + (0.2*0*1) = 0.4
1	1	1	1	0.2	0.4
0	0	0	0	0.2	0.4
0	1	1	1	0.2	0.4
1	0	1	1	0.2	0.4
1	1	1	1	0.2	0.4

* And function using perceptron rule solved example.

Perceptron limitation:

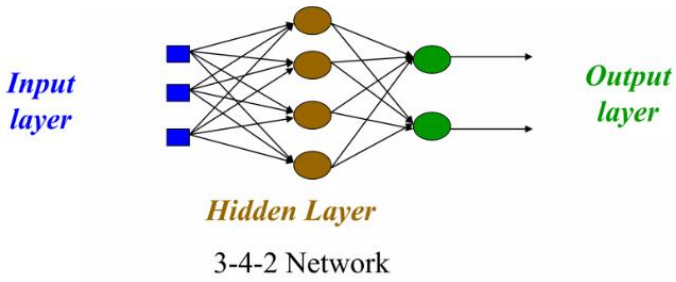
• The perceptron can only model linearly separable functions,
  – Those functions which can be drawn in 2-dim graph and single straight line separates values in two part.
• Boolean functions given below are linearly separable:
  – AND
  – OR
  – COMPLEMENT
• It cannot model XOR function as it is non-linearly separable.
  – When the two classes are not linearly separable, it may be desirable to obtain a linear separator that minimizes the mean squared error.

XOR: Non-linearly separable function:



Multi-layer Feed forward NN:

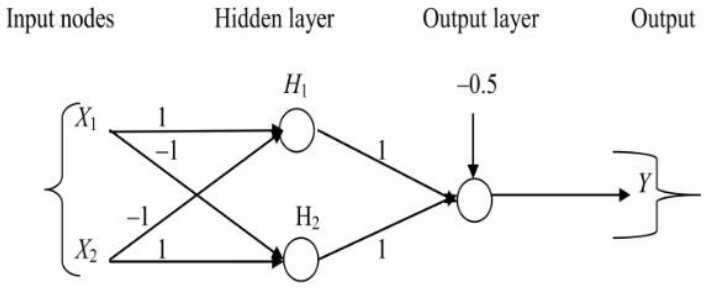
- FFNN is a more general network architecture, where there are hidden layers between input and output layers.
- Hidden nodes do not directly receive inputs nor send outputs to the external environment.
- FFNNs overcome the limitation of single-layer NN.
- They can handle non-linearly separable learning tasks.



3-4-2 Network

FFNN for XOR:

- The ANN for XOR has two hidden nodes that realizes this non-linear separation and uses the sign (step) activation function.
- Arrows from input nodes to two hidden nodes indicate the directions of the weight vectors (1,-1) and (-1,1).
- The output node is used to combine the outputs of the two hidden nodes.



Inputs		Output of Hidden Nodes		Output Node	X_1 XOR X_2
X_1	X_2	H_1	H_2		
0	0	0	0	–0.5 → 0	0
0	1	-1 → 0	1	0.5 → 1	1
1	0	1	-1 → 0	0.5 → 1	1
1	1	0	0	–0.5 → 0	0

Since we are representing two states by 0 (false) and 1 (true), we will map negative outputs (–1, –0.5) of hidden and output layers to 0 and positive output (0.5) to 1.

Gradient descent:

Gradient descent is an optimization algorithm used to minimize the loss function of a machine learning model.

Here's how gradient descent works:

1. **Initialization**: Initialize the model parameters (weights and biases) with some arbitrary values.
2. **Forward Pass**:
3. **Loss Computation**:
4. **Backpropagation**:
5. **Parameter Update**:

Back-propagation:
It searches for weight values that minimize the total error of the network over the set of training examples (training set).

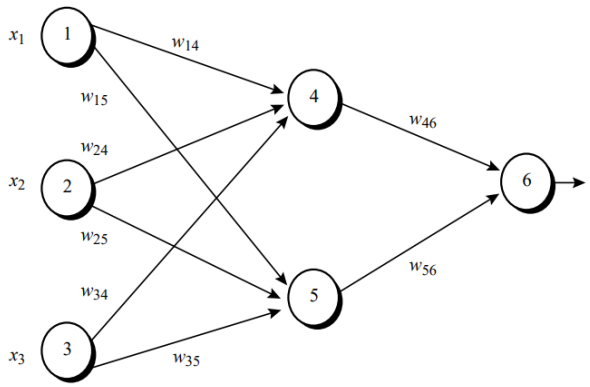
Back propagation consists of the repeated application of the following two passes:
Forward pass: In this step, pass a batch of input data through the model to compute the predicted outputs. Calculate the loss function, which measures the difference between the predicted outputs and the actual targets.

Backward pass: in this step the network error is used for updating the weights. The error is propagated backwards from the output layer through the network layer by layer. This is done by recursively computing the **local gradient** of each **neuron**.

Algorithm:

(1) Initialize all weights and biases in *network*;
(2) **while** terminating condition is not satisfied {
(3) **for** each training tuple $\boldsymbol{X}$ in D {
(4) // Propagate the inputs forward:
(5) **for** each input layer unit j {
(6) $O_j = I_j$; // output of an input unit is its actual input value
(7) **for** each hidden or output layer unit j {
(8) $I_j = \sum_i w_{ij} O_i + \theta_j$; //compute the net input of unit j with respect to the previous layer, i
(9) $O_j = \frac{1}{1+e^{-I_j}}$; } // compute the output of each unit j
(10) // Backpropagate the errors:
(11) **for** each unit j in the output layer
(12) $Err_j = O_j(1 - O_j)(T_j - O_j)$; // compute the error
(13) **for** each unit j in the hidden layers, from the last to the first hidden layer
(14) $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$; // compute the error with respect to the next higher layer, k
(15) **for** each weight w_{ij} in *network* {
(16) $\Delta w_{ij} = (l) Err_j O_i$; // weight increment
(17) $w_{ij} = w_{ij} + \Delta w_{ij}$; } // weight update
(18) **for** each bias θ_j in *network* {
(19) $\Delta \theta_j = (l) Err_j$; // bias increment
(20) $\theta_j = \theta_j + \Delta \theta_j$; } // bias update
(21) } }

Example 1:



Step 01:

Table 9.1: Initial input, weight, and bias values.

x_1	x_2	x_3	w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
1	0	1	0.2	−0.3	0.4	0.1	−0.5	0.2	−0.3	−0.2	−0.4	0.2	0.1

Step 02:

Table 9.2: The net input and output calculations.

Unit j	*Net input, I_j*	*Output, O_j*
4	$0.2 + 0 - 0.5 - 0.4 = -0.7$	$1/(1 + e^{0.7}) = 0.332$
5	$-0.3 + 0 + 0.2 + 0.2 = 0.1$	$1/(1 + e^{-0.1}) = 0.525$
6	$(-0.3)(0.332) - (0.2)(0.525) + 0.1 = -0.105$	$1/(1 + e^{0.105}) = 0.474$

Step 03:

Table 9.3: Calculation of the error at each node.

Unit j	*Err_j*
6	$(0.474)(1 - 0.474)(1 - 0.474) = 0.1311$
5	$(0.525)(1 - 0.525)(0.1311)(-0.2) = -0.0065$
4	$(0.332)(1 - 0.332)(0.1311)(-0.3) = -0.0087$

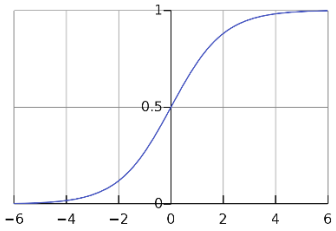
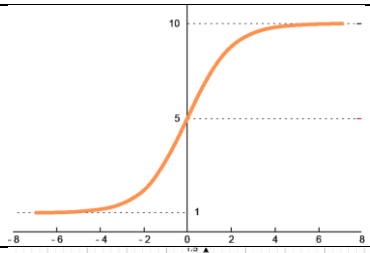
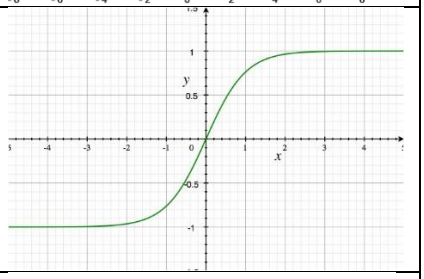
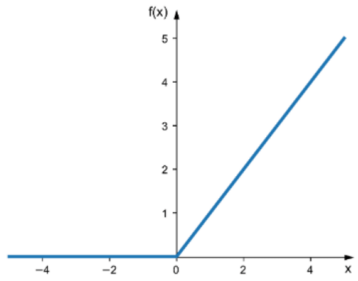
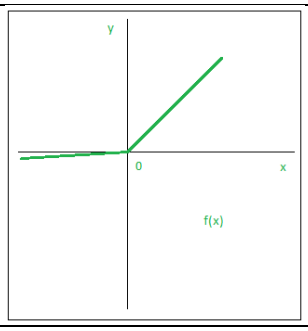
Step 04:

Table 9.4: Calculations for weight and bias updating.

Weight or bias	*New value*
w_{46}	$-0.3 + (0.9)(0.1311)(0.332) = -0.261$
w_{56}	$-0.2 + (0.9)(0.1311)(0.525) = -0.138$
w_{14}	$0.2 + (0.9)(-0.0087)(1) = 0.192$
w_{15}	$-0.3 + (0.9)(-0.0065)(1) = -0.306$
w_{24}	$0.4 + (0.9)(-0.0087)(0) = 0.4$
w_{25}	$0.1 + (0.9)(-0.0065)(0) = 0.1$
w_{34}	$-0.5 + (0.9)(-0.0087)(1) = -0.508$
w_{35}	$0.2 + (0.9)(-0.0065)(1) = 0.194$
θ_6	$0.1 + (0.9)(0.1311) = 0.218$
θ_5	$0.2 + (0.9)(-0.0065) = 0.194$
θ_4	$-0.4 + (0.9)(-0.0087) = -0.408$

Another FF with the weights and biases in the Table 9.4 if needed.
The old error was [1-0.474] = 0.526
After adjusting the weight the new error is 0.515

Activation Functions:

Activation F	Range	Equation	Figure
Sigmoid	0 to 1	$\sigma(x) = \frac{1}{1+e^{-x}} = \frac{e^x}{1+e^x}$	
Softmax	0 to 1	$s\left(x_i\right) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$	
Tanh	-1 to 1	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	
ReLU	0 to x	ReLU(x) = max(0,x)	
Leaky RELU		$f(x) = ax, x < 0$ $f(x) = x, \text{otherwise}$	

(i) Step Activation Function: The Step activation function is used in the perceptron network. This is usually used in single-layer networks to convert to an output that is binary (0 or 1) or Bipolar (-1 or 1). These are called Binary Step Function and Bipolar Step Function Respectively. Here if the input value to a function is greater than a threshold value then the output is 1 (neuron will fire) otherwise 0 or -1 in the case of Bipolar Step Function (neuron will not fire).

Some Problems in Adding more hidden layers:

Vanishing Gradient Problem:

- **Cause**: When backpropagating gradients through many layers, the gradients can become extremely small as they are multiplied by the weights of each layer during backpropagation. If the gradients become too small, they can effectively "vanish," meaning they don't contribute significantly to updating the weights of earlier layers. This is why we don't use the sigmoid activation function. Activation functions like RELU and its variants don't cause small derivatives.

- **Effect**: The earlier layers of the network learn very slowly or not at all, leading to slow convergence or poor performance.

Exploding Gradient Problem

- **Cause**: Conversely, when backpropagating gradients through many layers, the gradients can become extremely large as they are multiplied by the weights of each layer during backpropagation. If the gradients become too large, they can lead to large updates in the weights, causing instability and convergence issues.

- **Effect**: Unstable training, diverging loss, and difficulty in converging to a good solution.

[] https://ysu1989.github.io/courses/sp20/cse5243/Han_Chapter09.pdf

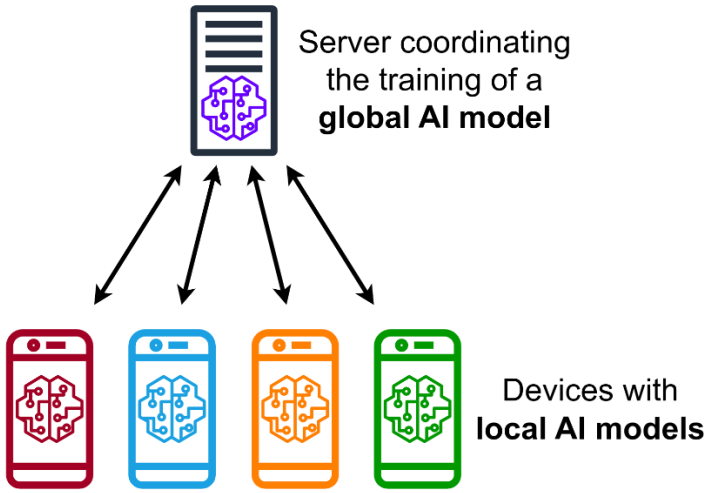
Federated learning:
Federated Learning is simply the **decentralized** form of Machine Learning, works by training a central model across decentralized devices or servers. Instead of moving all data to a central location, the model is trained locally on each device, and only the model updates are shared. This maintains privacy and allows **collaborative learning** without sharing raw data.

1. Deploy the machine learning model to local copies on user devices.

2. The model learns and improves over time based on user input.

3. The devices are then allowed to transfer the training **results**, from the local copy back to the central server.

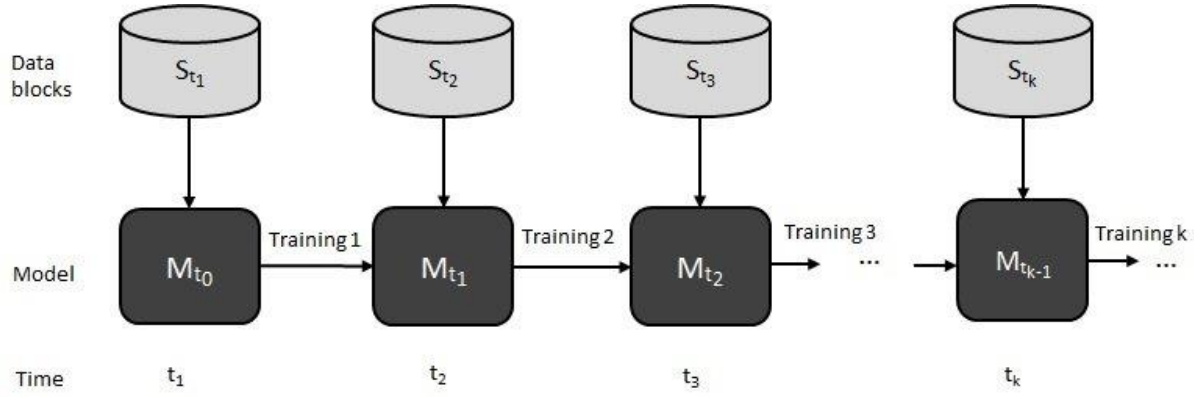
4. The results will be aggregated together in the centralized server and updates its central machine learning model.



Incremental Learning:
Incremental learning is a methodology of machine learning where an AI model learns and enhances its knowledge progressively, without forgetting previously acquired information. In contrast, in traditional batch learning, the machine learning model is trained on the entire data set at once.

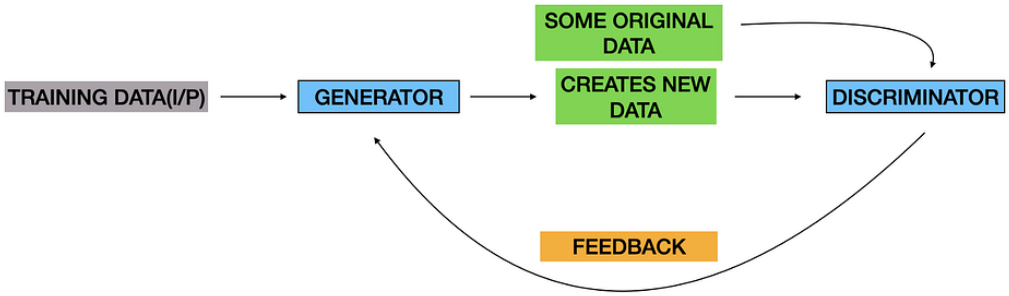
For instance, consider a spam email filter. With batch learning, the filter is trained with a large set of emails at once and then applied to future emails. If the nature of spam emails changes, the filter might start fail unless retrained on a new batch of emails.

On the other hand, an incremental learning-based spam filter could potentially learn to recognize new spam styles without needing a whole new batch of training data.



Data Stk is not presented priorly but arrives over a while.
The task is to infer a trusted model Mt after every time step based on the example **Stk** and the previous model **Mtk-1** only.

Generative AI:
Generative AI refers to artificial intelligence systems that have the capability to generate new, original content. These systems can create data, such as images, text, audio, or other types of media, that was not explicitly present in their training data. Generative AI techniques include generative adversarial networks (GANs), auto-encoders, and deep reinforcement learning. Applications of generative AI range from generating realistic images and deep-fake videos to creating human-like text and even developing new drugs in the pharmaceutical industry. It's a rapidly evolving field with both **creative and practical** uses.



GANs consist of two neural networks, the generator and the discriminator.
Generator: The generator in a GAN is a neural network responsible for generating synthetic data samples that resemble real data. It takes random noise as input and learns to generate data that is indistinguishable from real data.

Discriminator: The discriminator is another neural network that evaluates the authenticity of the data samples. It distinguishes between real data samples from the training set and fake data samples generated by the generator.

During training, the generator aims to produce data that fools the discriminator into classifying it as real.

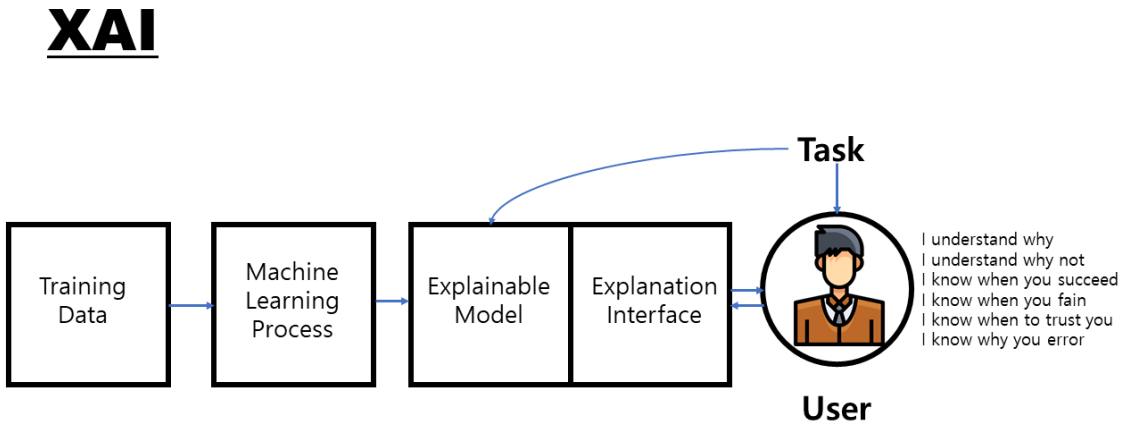
Explainable AI

Explainable AI (XAI) refers to the capability of artificial intelligence systems to provide understandable explanations or justification for their decisions and actions. It aims to enhance transparency and trust in AI systems by making their inner workings interpretable to human.

Model interpretability can be examined in two levels:

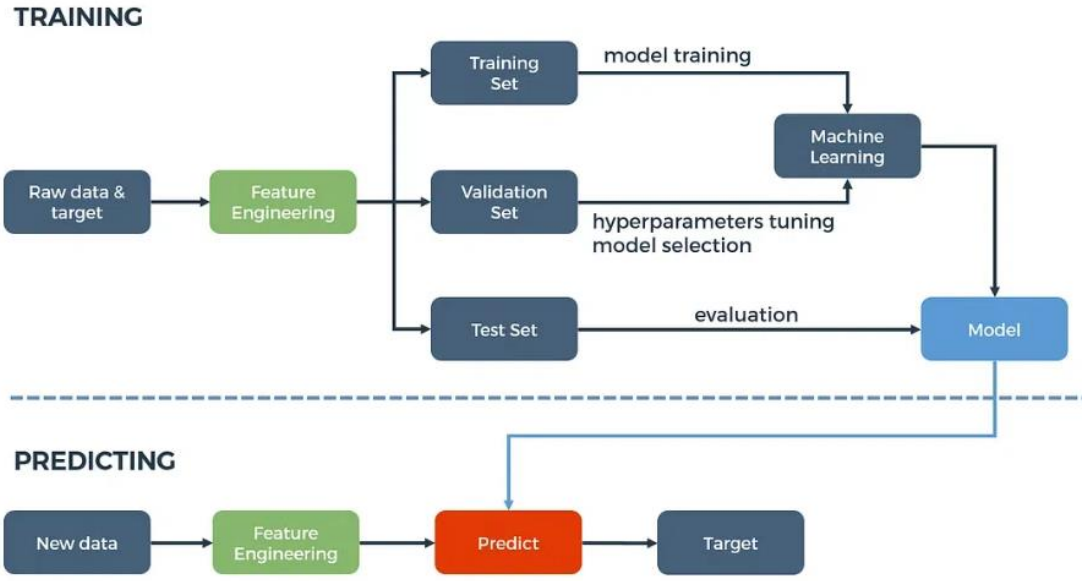
- **Global Interpretation:** Examines the model from a broader perspective. For example, let's say we are working on a house price dataset and we implemented a neural network. The global interpretation might say "Your model uses # of squared feet as an important feature to derive predictions.

- **Local Interpretation**: As the name suggests, this approach is focused on a certain observation/data point. Let's continue moving forward with our example. Prediction for a really small house turned out large. Local interpretation looks at the other features and it might say "Your model predicted this way because the location of the house is very close to the city center.



AutoML
AutoML, short for automated machine learning, is the process of automating various machine learning model development processes. The goal is to make machine learning more accessible for individuals and organizations with limited expertise in data science and machine learning.

AutoML Cycle:



AutoML Systems:
AutoML systems can automate several steps involved in building a machine-learning model. Some of them are as follows:

- **Data preparation:** Handling missing values, scaling features, encoding categorical variables, and splitting data into training and testing sets.
- **Feature engineering:** Extracting useful features from raw data.
- **Model selection & Training**: Choosing an appropriate machine learning algorithm and hyperparameters.
- **Hyperparameter tuning**: Fine-tuning the settings of a machine learning model to optimize performance.
- **Ensemble modelling**: Combining multiple models to improve performance.
- **Deployment:** Putting a trained model into production
- **Monitoring and Maintenance**: After the deployement model can be maintained and monitered based the new data.

Transfer Learning:
Transfer learning is a technique in machine learning where a model trained on one task is used as the starting point for a model on a second task. This can be useful when the second task is similar to the first task.

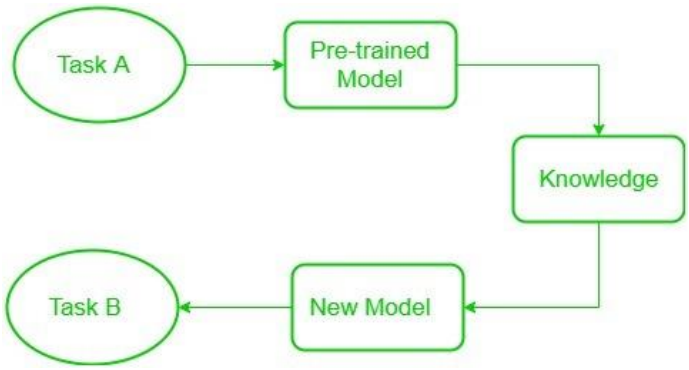
Why Transfer Learning:
Many deep neural networks trained on images have a common phenomenon: in their early layers, they learn low-level features regardless of a particular dataset. Thus, learning these features in one task of detecting lions can be used in other tasks like detecting humans.

How does Transfer Learning work?
Pre-trained Model: Start with a model that has previously been trained for a certain task using a large set of data. This model has identified general features and patterns relevant to numerous related jobs. This model is also known as base model.

Transfer Layers: In the pre-trained model, find a set of layers that capture generic information relevant to the new task as well as the previous one. These layers are frequently found near the top of the network.

Fine-tuning: Retrain the chosen layers while enabling the model to modify its parameters to better suit the demands of the current task.

The Block diagram:



Transformer

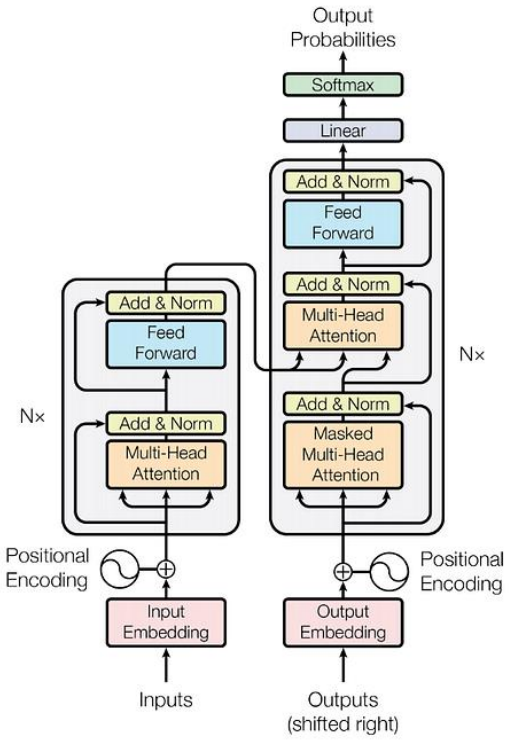


Figure 1: The Transformer - model architecture.

The Transformer is a deep learning architecture introduced in the paper "Attention is All You Need" by Vaswani et al. It revolutionized the field of natural language processing (NLP) by introducing a novel mechanism called self-attention.

Self-Attention Mechanism: The core of the Transformer architecture is the self-attention mechanism, which enables the model to weigh the importance of different words in a sentence.

Encoder-Decoder Architecture: The Transformer consists of an encoder and a decoder. The encoder map an input sequence to a sequence of continuous representations. Decoder receives the output of the encoder together with the decoder output at the previous time step to generate an output sequence. Both the encoder and decoder consist of multiple layers of self-attention mechanisms and feed-forward neural networks.

Positional Encoding: positional encoding is added to the input embeddings to provide information about the position of each word in the sequence.

Feed-forward network: The feed-forward network accepts attention vectors one at a time. And the best thing here is, unlike the case of the RNN, each of these attention vectors is independent of one another. So, we can apply parallelization here.

BERT
Bidirectional Encoder Representations from Transformers (BERT) is a language model based on the transformer architecture. It was introduced in 2018 by researchers at Google.

BERT was originally implemented in the English language at two model sizes: (1) $\text{BERT}_{\text{BASE}}$: 12 encoders with 12 bidirectional self-attention heads totaling 110 million parameters, and (2) $\text{BERT}_{\text{LARGE}}$: 24 encoders with 16 bidirectional self-attention heads totaling 340 million parameters. Both models were pre-trained on the Toronto BookCorpus and English Wikipedia.

BERT uses WordPiece to convert each English word into an integer code. Its vocabulary has size 30,000. Any token not appearing in its vocabulary is replaced by [UNK] for "unknown".

BERT was pre-trained simultaneously on two tasks:
Masked Language Model (MLM): BERT randomly masks some of the input tokens and trains the model to predict the masked tokens based on the surrounding context.

Next Sentence Prediction (NSP): BERT predicts whether a pair of sentences appears consecutively in the original text.

- For example, given "[CLS] my dog is cute [SEP] he likes playing" the model should output token [IsNext].
- Given "[CLS] my dog is cute [SEP] how do magnets work" the model should output token [NotNext].

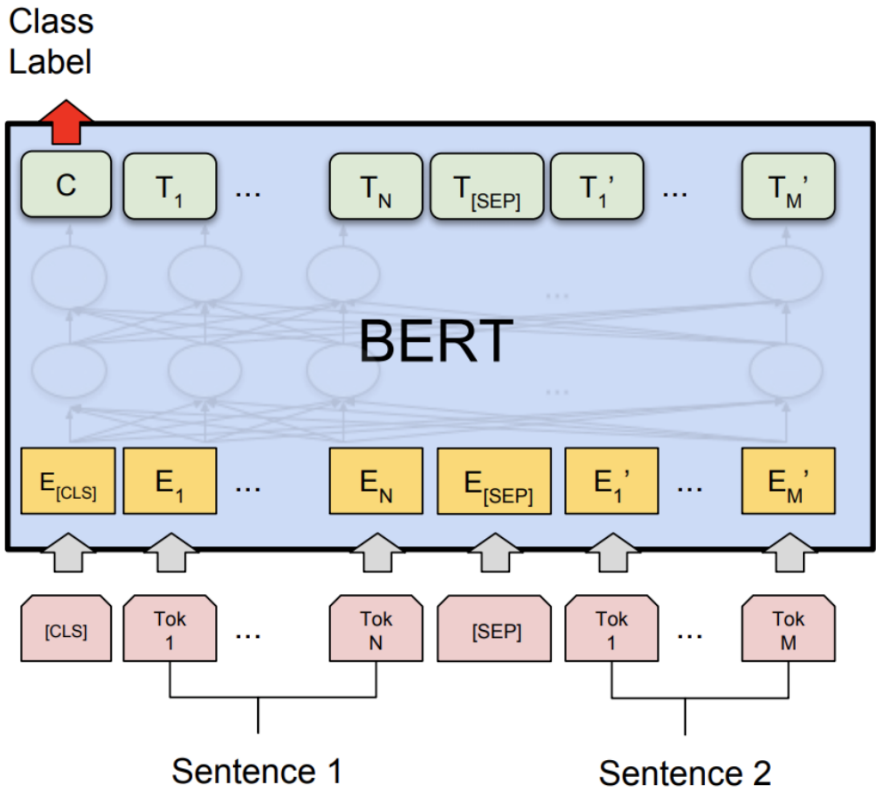
Input Representation: BERT accepts tokenized input sequences, where each token is represented by its corresponding embedding vector. Special tokens are added to mark the beginning ([CLS]) and separation ([SEP]) of sentences. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

The representation vectors move through 12/24 Transformer encoders, then they are un-embedded by an affine-Add & LayerNorm-linear.

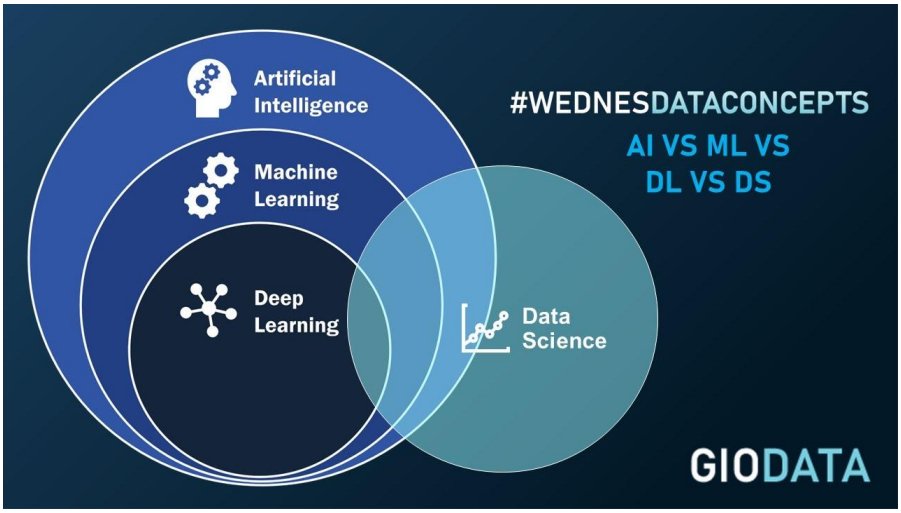
Output Layers:

- For classification tasks, the representation of the special [CLS] token at the beginning of the sequence is used as the aggregate representation of the entire input sequence.

- The [CLS] token representation is fed into additional layers (e.g., a fully connected layer) for task-specific classification or regression.



AI vs ML vs DL vs DS:



Artificial Intelligence (AI):

- AI refers to the broader concept of machines performing tasks that typically require human intelligence.
- It encompasses various techniques, including machine learning and deep learning, to enable machines to mimic cognitive functions such as learning, problem-solving, perception, and decision-making.
- AI systems can range from simple rule-based systems to complex neural networks.

Machine Learning (ML):

- ML is a subset of AI that focuses on enabling machines to learn from data without explicit programming.
- ML algorithms can be categorized into supervised learning, unsupervised learning, and reinforcement learning.

Deep Learning (DL):

- Deep learning is a subset of ML that involves neural networks with multiple layers (deep architectures) to learn intricate patterns from data.
- Common DL architectures include convolutional neural networks (CNNs), recurrent neural networks (RNNs), and transformer.

Data Science (DS):

- Data science is a multidisciplinary field that encompasses tasks such as data preprocessing, feature engineering, model building, evaluation, and deployment.
- It combines programming languages (e.g., Python, R), tools (e.g., TensorFlow, PyTorch), and methodologies.

TensorFlow vs PyTorch vs Keras:

	TensorFlow	**PyTorch**	**Keras**
Introduction	TensorFlow is an open-source machine learning framework developed by Google Brain.	PyTorch is an open-source machine learning framework developed by Facebook's AI Research lab (FAIR).	Keras is an open-source neural network library that has been integrated into TensorFlow as its high-level API
API Level	High and Low	Low	High
Architecture	Not easy to use	Complex, less readable	Simple, concise, readable
Debugging	Difficult to conduct debugging	Good debugging capabilities	Simple network, so debugging is not often needed
Speed	Fast, high-performance	Fast, high-performance	Slow, low performance
Written In	C++, CUDA, Python	Lua	Python

Common terminology in ML: